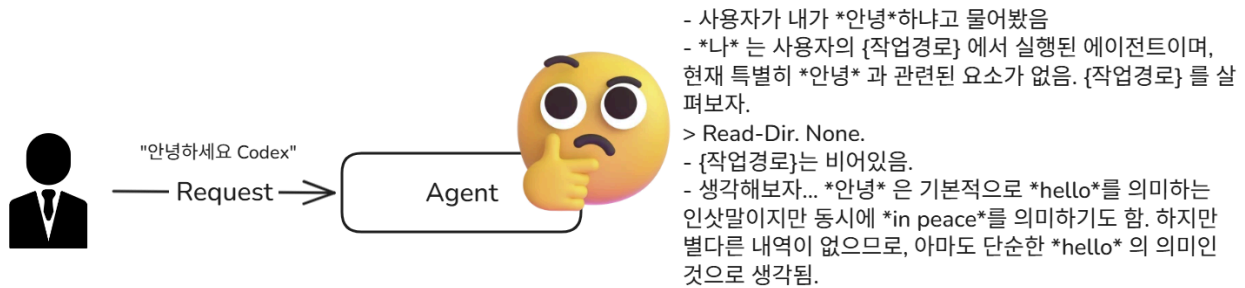


Week 3 CLI; Subprocess



The essence of programming

- **Coding**(Implement / Development)은 SDLC 단계 중 특정 단계의 극히 일부 활동에 불과
- 이제 AI는 코드를 상당히 정확하고 빠르게 생성하지만, 그것만으로 좋은 시스템이 나오지는 않음

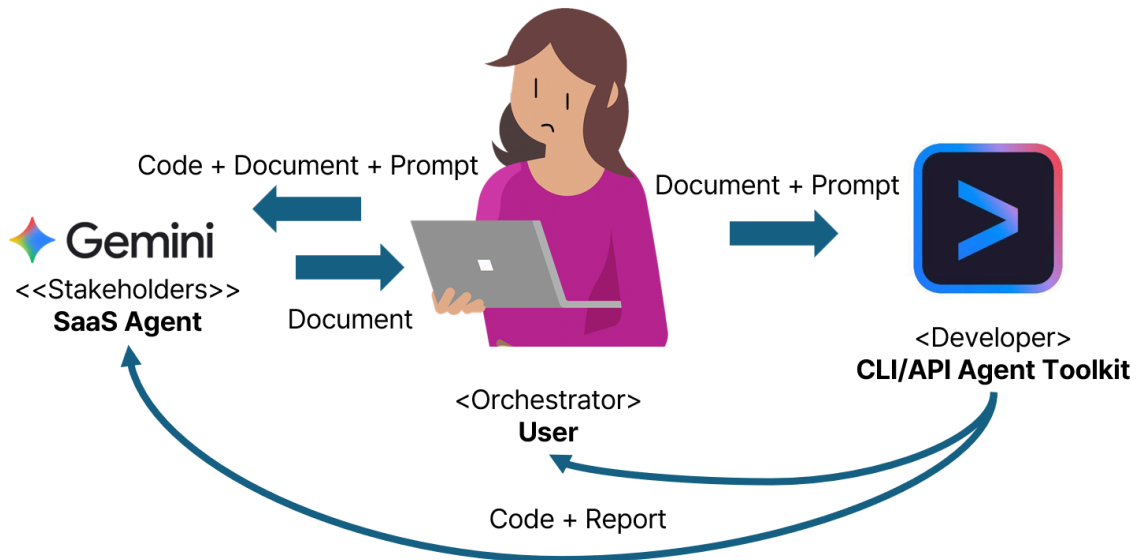
Failure Case

Sequential Agent 실패 사례 (Quick Sort).

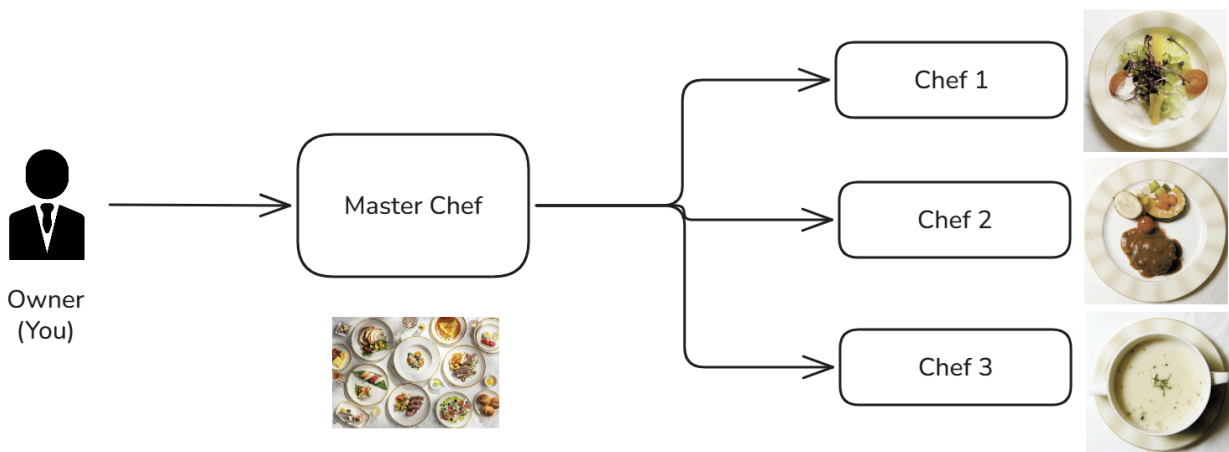
Think about pipeline → Vibe Coding and Agentic Coding

| 단일 도구에서 하나의 시스템으로의 전환

[기존 바이브 코딩]



[에이전트 코딩]

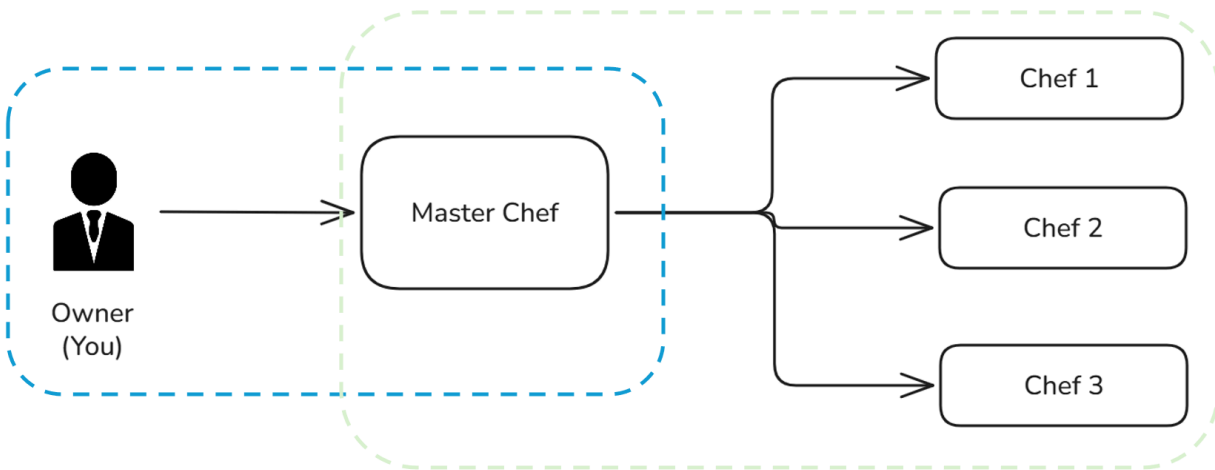


Thinking : System Requests of Restaurant

- **Project Sponsor (You)**
- **Business Need**
- **Business Requirements**

- Business Value
- Special Issues or Constraints

Two Axis of Agentic Coding Interface



Permissions of Agent

- > Codex, Codex CLI의 Github 경로를 찾아 실행 폴더에 Clone하고, User Request를 Codex가 어떻게 분해하는지 확인하여 마크다운 파일로 저장해주세요.
- 공식 저장소는 <https://github.com/openai/codex>로 확인했습니다. 이제 저장소를 현재 폴더에 가져오고, 내부 문서와 소스에서 요청 분해 흐름이 실제로 어디에 정의되어 있는지 추적하겠습니다.
 - Updated Plan
 - ✓ Verify the official Codex CLI GitHub repository URL
 - Clone the repository into the current working directory
 - Analyze how Codex decomposes user requests using available source/docs
 - Write findings to a markdown file in the workspace
 - Running `git clone https://github.com/openai/codex.git`
| `codex-cli`

Would you like to run the following command?

Reason: Do you want to allow cloning the official Codex repository into this workspace so I can analyze it and write the markdown summary?

```
$ git clone https://github.com/openai/codex.git
codex-cli
```

- › 1. Yes, proceed (y)
- 2. Yes, and don't ask again for commands that start with `git clone https://github.com/openai/codex.git codex-cli` (p)
- 3. No, and tell Codex what to do differently (esc)

Remind :: Permissions

```
claude --dangerously-skip-permissions
gemini --yolo
codex --dangerously-bypass-approvals-and-sandbox
```

권한 수준이 높으면 단점: **사용자가 더이상 완전한 제어권을 가지지 않음. (멈춰야 할 순간에 멈추지 않음.)**

예시: Code 구현에 대한 테스트 과정에서, 장시간 동안 복잡한 구성의 Test Code를 작성하여 테스트 한 뒤, 사용자의 의도에 상관없이 테스트 완료 후의 산출물, 그리고 테스트용 코드를 복구 불가능하게 삭제함

Session continue / resume

claude -c : 최근 세션 즉시 이어감 (—continue)

claude --resume : 세션 목록에서 선택

claude --resume <session-id> : 특정 세션으로 직접 이어감

codex resume : 세션 picker에서 선택

codex resume <session-id> : 특정 세션으로 직접 이어감

codex resume --last : 최근 세션 즉시 이어감 (현재 디렉토리 기준)

gemini --resume : 최근 세션 이어감

gemini --resume <index> : 인덱스로 특정 세션 이어감

gemini --list-sessions : 세션 목록 확인

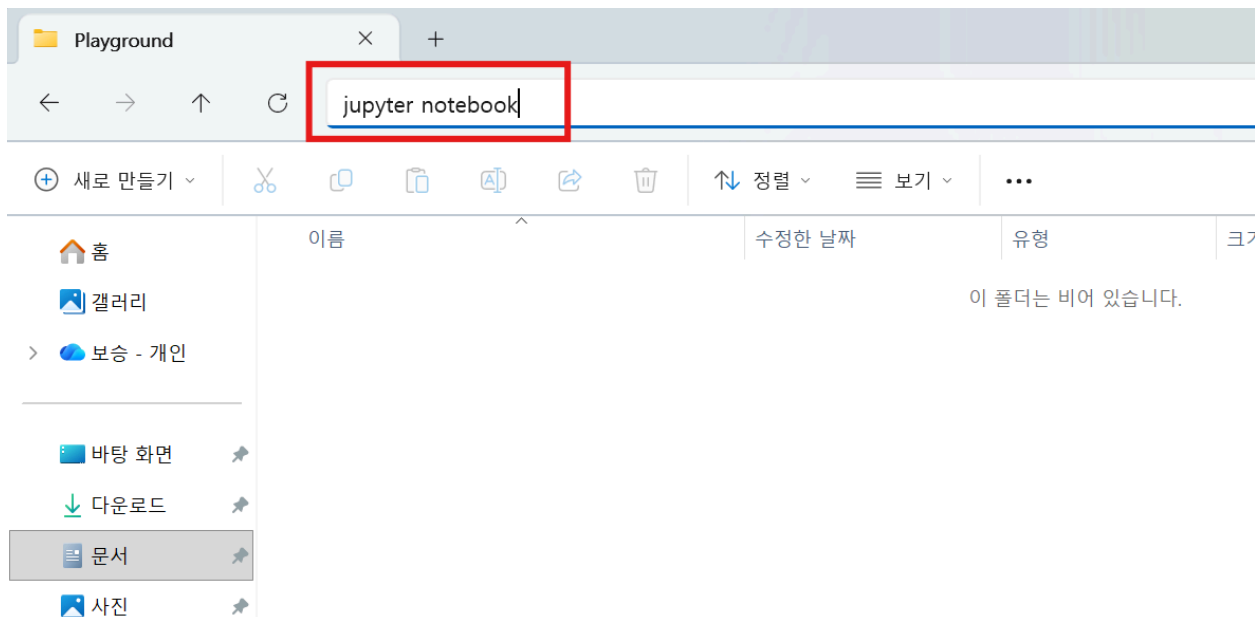
```
[1]: import subprocess
import sys
import os
import time
import threading
from dataclasses import dataclass
from typing import Literal, Optional

print(f"Python: {sys.version}")
print(f"OS: {os.name} ({sys.platform})")
print(f"CWD: {os.getcwd()}")
print("\n환경 준비 완료.")

Python: 3.12.10 (tags/v3.12.10:0cc8128, Apr 8 2025, 12:21:36) [MSC v.1943 64 bit (AMD64)]
OS: nt (win32)
CWD: C:\Termis\CSE3308_Advanced_agent

환경 준비 완료.
```

Week3 강의 시작전 다음 요소가 설치되어 있어야 합니다.
- jupyter



- 또는 디렉터리의 빈 공간 우클릭 > 터미널에서 열기(cmd 확인) > jupyter notebook ..

CLI 호출의 두 가지 방식

터미널에서 직접 실행할 때와 Python 코드에서 실행할 때, 프롬프트 전달 방식이 다름

방식 1: 명령줄 인자 (터미널에서 직접 실행)

```
claude -p "1+1의 답을 숫자만 말해줘"
```

프롬프트가 명령어의 일부로 전달됨

방식 2: stdin 파이프 (Python에서 실행)

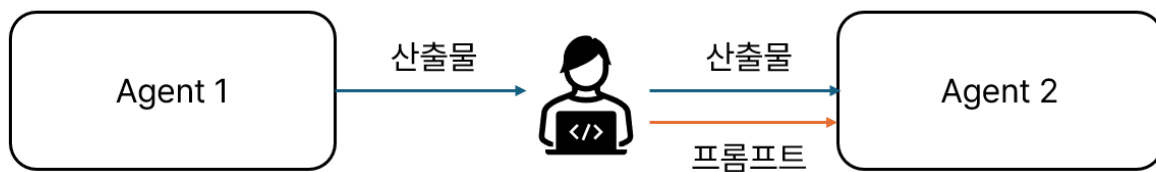
```
# Claude
subprocess.run(["claude", "-p"], input="1+1의 답을 숫자만 말해줘", capture_output=True, text=True)

# Codex (darwin에서는 .cmd 제거)
subprocess.run(["codex.cmd", "exec"], input="1+1의 답을 숫자만 말해줘", capture_output=True, text=True)

# Gemini (darwin에서는 .cmd 제거)
subprocess.run(["gemini.cmd", "-p", "1+1의 답을 숫자만 말해줘"], capture_output=True, text=True)
```

프롬프트가 **stdin**(표준 입력)으로 전달됨

Subprocess



User가 두 에이전트 간 대화를 시키고 싶을 때

subprocess.run() 파라미터 정리

```

subprocess.run(
    cmd,                                # 실행할 명령 리스트 (예: ["claude", "-p"])
    input=prompt,                       # stdin으로 보낼 문자열 (프롬프트)
    capture_output=True,               # stdout/stderr를 캡처
    encoding="utf-8",                 # 바이트 대신 문자열로 반환
    timeout=120,                       # 최대 대기 시간 (초)
)

```

파라미터	역할
<code>input</code>	프로세스의 stdin에 문자열을 보내고, 보낸 뒤 stdin을 자동으로 닫는다
<code>capture_output</code>	<code>stdout=PIPE, stderr=PIPE</code> 의 축약형
<code>text</code>	입출력을 <code>bytes</code> 대신 <code>str</code> 로 처리 → <code>encoding="utf-8"</code> 로 오버라이딩
<code>timeout</code>	지정 시간 초과 시 <code>TimeoutExpired</code> 예외 발생

Initialize

```

import subprocess
import sys
import os
import time
import threading
import json

print(f"Python: {sys.version}")
print(f"OS: {os.name} ({sys.platform})")
print(f"CWD: {os.getcwd()}")
print("\n환경 준비 완료.")

```

Step 1

```

MY_AGENT = "claude"

def check_cli(agent: str) -> bool:
    """CLI가 설치되어 있는지 확인한다."""
    binary = agent
    if agent == "codex" and os.name == "nt":

```

```

        binary = "codex.cmd"
    elif agent == "gemini" and os.name == "nt":
        binary = "gemini.cmd"

    try:
        result = subprocess.run(
            [binary, "--version"],
            capture_output=True, text=True, timeout=10
        )
        version = result.stdout.strip() or result.stderr.strip()
        print(f" {agent}: {version}")
        return True
    except FileNotFoundError:
        print(f" {agent}: 설치되지 않음")
        return False
    except Exception as e:
        print(f" {agent}: 확인 실패 ({e})")
        return False

check_cli(MY_AGENT)

```

Subprocess

build_cli_command()로 명령을 만들고, subprocess.run()으로 실행

Step 2

```

def pick_binary(agent: str) -> str:
    """에이전트 이름 → 실행 바이너리 이름"""
    if agent == "codex":
        return "codex.cmd" if os.name == "nt" else "codex"
    if agent == "gemini":
        return "gemini.cmd" if os.name == "nt" else "gemini"
    if agent == "claude":
        return "claude"
    raise ValueError(f"Unknown agent: {agent}")

```



```

def build_cli_command(agent: str, prompt: str | None = None) -> list
[str]:
    """
    에이전트 이름을 받아 oneshot CLI 명령 리스트를 반환한다.
    Gemini는 -p에 프롬프트가 필수이므로 prompt를 명령에 포함한다.
    Claude/Codex는 stdin으로 전달하므로 prompt를 명령에 포함하지 않는다.
    """
    exe = pick_binary(agent)

    if agent == "codex":
        # --skip-git-repo-check: Git 저장소 밖에서도 실행 가능
        return [exe, "exec", "--ephemeral", "--json", "--skip-git-re
po-check"]

    if agent == "gemini":
        # -p 뒤에 프롬프트 문자열이 필수 (stdin으로 전달 불가)
        return [exe, "-p", prompt or "", "--output-format", "json"]

    if agent == "claude":
        return [exe, "-p", "--no-session-persistence", "--output-for
mat", "json"]

    raise ValueError(f"Unknown agent: {agent}")

def run_agent(agent: str, prompt: str, timeout: int = 120) -> subprocess.CompletedProcess:
    """
    에이전트를 호출하고 결과를 반환한다.
    Claude/Codex: 프롬프트를 stdin으로 전달.
    Gemini: 프롬프트를 명령줄 인자로 전달.
    """
    cmd = build_cli_command(agent, prompt)
    # Gemini는 프롬프트가 이미 cmd에 포함 → stdin 불필요
    use_stdin = prompt if agent != "gemini" else None
    return subprocess.run(
        cmd,
        input=use_stdin,
        capture_output=True,
        encoding="utf-8",
    )

```

```

        timeout=timeout,
    )

```

— 사용 예시 —

```

example_cmd = build_cli_command(MY_AGENT)
print(example_cmd)

```

Step 3 - Execute

subprocess.CompletedProcess — 반환값

subprocess.run() 은 CompletedProcess 객체를 반환

```

result = run_agent(MY_AGENT, "1+1의 답을 숫자만 말해줘")

```

```

result.returncode # 종료 코드 (0이면 성공, 그 외는 실패)
#result.stdout   # 표준 출력 (CLI가 출력한 응답)
#result.stderr    # 표준 에러 (에러 메시지나 경고)
#result.args      # 실행한 명령 리스트 (디버깅용)

```

속성	타입	의미	예시
.returncode	int	프로세스 종료 코드	0 = 성공, 1 = 에러
.stdout	str	CLI가 출력한 내용	"2"
.stderr	str	에러/경고 메시지	"" (정상 시 비어 있음)
.args	list	실행된 명령어	["claude", "-p"]

returncode == 0 이면 성공이라는 규칙은 Unix/Windows 공통이며, 모든 CLI 도구가 이 규칙을 따르므로, result.returncode 만 확인하면 성공 여부를 알 수 있음

```
[6]: result = run_agent(MY_AGENT, "1+1의 답을 숫자만 말해줘")

[7]: result

[7]: CompletedProcess(args=['gemini.cmd', '-p', '1+1의 답을 숫자만 말해줘', '--output-format', 'json'], returncode=0, stdout='{
  "session_id": "d9d37a0a-e6b
e-486c-be34-dac0c3ab27b2",
  "response": "2",
  "stats": {
    "models": {
      "gemini-2.5-flash-lite": {
        "api": {
          "totalRequ
ests": 1,
          "totalErrors": 0,
          "totalLatencyMs": 1275,
          "tokens": {
            "input": 2988,
            "prompt": 2988,
            "candidates": 26,
            "total": 3168,
            "cached": 0,
            "thoughts": 154,
            "tool": 0
          }
        },
        "roles": {
          "utility_router": {
            "totalRequests": 1,
            "totalErrors": 0,
            "totalLatencyMs": 1275,
            "tokens": {
              "input": 2988,
              "prompt": 2988,
              "candidates": 26,
              "total": 3168,
              "cached": 0,
              "thoughts": 154,
              "tool": 0
            }
          }
        },
        "gemini-3-flash-preview": {
          "api": {
            "totalRequests": 1,
            "totalErrors": 0,
            "totalLatencyMs": 2718,
            "tokens": {
              "input": 7464,
              "prompt": 7464,
              "candidates": 1,
              "total": 7524,
              "cached": 0,
              "thoughts": 59,
              "tool": 0
            }
          },
          "roles": {
            "main": {
              "totalRequests": 1,
              "totalErrors": 0,
              "totalLatencyMs": 2718,
              "tokens": {
                "input": 7464,
                "prompt": 7464,
                "candidates": 1,
                "total": 7524,
                "cached": 0,
                "thoughts": 59,
                "tool": 0
              }
            }
          },
          "tools": {
            "totalCalls": 0,
            "totalSuccess": 0,
            "totalFail": 0,
            "totalDurationMs": 0,
            "totalDecisions": {
              "accept": 0,
              "reject": 0,
              "modify": 0,
              "auto_accept": 0,
              "byName": {},
              "files": {},
              "totalLinesAdded": 0,
              "totalLinesRemoved": 0
            }
          }
        }
      }
    }
  },
  stderr="Loaded cached credentials.\n"
}
```

```
[12]: #result.returncode # 종료 코드 (0이면 성공, 그 외는 실패)
#result.stdout # 표준 출력 (CLI가 출력한 응답)
#result.stderr # 표준 에러 (에러 메시지나 경고)
result.args # 실행한 명령 리스트 (디버깅용)
```

```
[12]: ['gemini.cmd', '-p', '1+1의 답을 숫자만 말해줘', '--output-format', 'json']
```

Output Example

```
{
  "type": "result",
  "subtype": "success",
  "is_error": false,
  "duration_ms": 3063,
  "duration_api_ms": 3009,
  "num_turns": 1,
  "result": "\n\n2",
  "stop_reason": "end_turn",
  "session_id": "3a54ec45-3a23-400f-b60b-93debebc51c4",
  "total_cost_usd": 0.045919,
  "usage": {
    "input_tokens": 3,
    "cache_creation_input_tokens": 6624,
    "cache_read_input_tokens": 8758,
    "output_tokens": 5,
    "server_tool_use": {
      "web_search_requests": 0,
      "web_fetch_requests": 0
    },
    "service_tier": "standard",
    "cache_creation": {
      "ephemeral_1h_input_tokens": 6624,
      "ephemeral_5m_input_tokens": 0
    },
  },
}
```

```

    "inference_geo": "",
    "iterations": [],
    "speed": "standard"
  },
  "modelUsage": {
    "claude-opus-4-6[1m]": {
      "inputTokens": 3,
      "outputTokens": 5,
      "cacheReadInputTokens": 8758,
      "cacheCreationInputTokens": 6624,
      "webSearchRequests": 0,
      "costUSD": 0.045919,
      "contextWindow": 1000000,
      "maxOutputTokens": 32000
    }
  },
  "permission_denials": [],
  "fast_mode_state": "off",
  "uuid": "33a14fd1-9e09-463c-bb66-9036b7457b80"
}

```

Json load

```

# Gemini, Claude
response = json.loads(result.stdout)
response

# Codex
lines = [json.loads(line) for line in result.stdout.strip().splitlines()]
lines

```

```
[14]: response = json.loads(result.stdout)

[15]: response

[15]: {'session_id': 'd9d37a0a-e6be-486c-be34-dac0c3ab27b2',
      'response': '2',
      'stats': {'models': {'gemini-2.5-flash-lite': {'api': {'totalRequests': 1,
                    'totalErrors': 0,
                    'totalLatencyMs': 1275},
                    'tokens': {'input': 2988,
                                'prompt': 2988,
                                'candidates': 26,
                                'total': 3168,
                                'cached': 0,
                                'thoughts': 154,
                                'tool': 0}},
                  'roles': {'utility_router': {'totalRequests': 1,
                    'totalErrors': 0,
                    'totalLatencyMs': 1275,
                    'tokens': {'input': 2988,
                                'prompt': 2988,
                                'candidates': 26,
                                'total': 3168,
                                'cached': 0,
                                'thoughts': 154,
                                'tool': 0}}}},
      'gemini-3-flash-preview': {'api': {'totalRequests': 1,
```

- Gemini의 재밌는 점: 이렇게 호출하면 gemini-2.5-flash-lite이 먼저 라우팅을 하고, 그 이후에 다른 모델을 재호출

response dict

```
{
  "session_id" : str,
  "input"      : str,
  "result"     : str
}
```

- Claude, Codex, Gemini의 응답 JSON Format이 모두 다릅니다. result의 .args와, result.stdout 을 분석하여 dict를 하나 만듭니다.

Expected Result:

```
[41]: parsed
```

```
[41]: {'session_id': '841f676a-cb80-4590-b885-351656184c57',  
      'input': '1+1의 답을 숫자만 말해줘',  
      'result': '2'}
```

Session Control

```
SESSION_ID = "841f676a-cb80-4590-b885-351656184c57"
```

- 이전 세션의 Session ID만 가져옵니다.

```
# Claude  
subprocess.run(["claude", "-p", "--resume", SESSION_ID], input="이전  
질문을 확인해주세요.", capture_output=True, encoding="utf-8")  
  
# Codex (darwin에서는 .cmd 제거)  
subprocess.run(["codex.cmd", "exec", "resume", SESSION_ID, "--skip-g  
it-repo-check", "이전 질문을 확인해주세요."], capture_output=True, encod  
ing="utf-8")  
  
# Gemini (darwin에서는 .cmd 제거)  
subprocess.run(["gemini.cmd", "--resume", SESSION_ID, "-p", "이전 질문  
을 확인해주세요."], capture_output=True, encoding="utf-8")
```

- Claude, Codex는 Session ID 영속성을 제거하는 플래그 "--ephemeral", "--no-session-persistence" 를 입력하였었고, Gemini는 모든 세션은 로컬에 저장되는 것이 원칙

Fix build_cli_command

```
if agent == "codex":  
    return [exe, "exec", "--json", "--skip-git-repo-check"]
```

```
if agent == "claude":  
    return [exe, "-p", "--output-format", "json"]
```

- 코텍스의 경우 "--ephemeral", 클로드의 경우 "--no-session-persistence" 제거

Restart Pipeline

```
result2 = subprocess.run(["claude", "-p", "--resume", SESSION_ID, "-  
-output-format", "json"], input="이전 질문을 확인해주세요.", capture_out  
put=True, encoding="utf-8")
```

Thinking

- 제공된 Code만으로는 Local 경로의 File을 에이전트에게 전달할 수 없습니다. AI와의 논의를 통해 어떻게 내 Local 경로의 File을 읽도록 할 수 있을지 논의해보세요.
- CLI에서 권한을 전부 허용하고 cse3308_easy_agent를 분석하여 각 Tab이 어떻게 하드코딩 되어 있고, Session ID를 관리하며, 다음 대화를 이어 나갈 수 있는지 알아보세요.

Discussion

- AI와의 논의를 통해 Agentic Coding에서 세션의 연속성을 관리해야 할 필요성에 대해 알아보세요.
- 하나의 작업 환경에서 각 Agent를 어떻게 관리할 수 있을지 고민해봅시다.